Tradução PT-BR — Capítulo 4

Arquivo original: TG2_Bruno_Silveira.pdf

Capítulo traduzido: 4 B.A.L.A.S Design and Implementation.

4 PROJETO E IMPLEMENTAÇÃO DO B.A.L.A.S

Este capítulo descreve todo o processo de desenvolvimento da ferramenta B.A.L.A.S. A Seção 4.1 apresenta uma visão geral do projeto, com os módulos de alto nível e a arquitetura de software; a Seção 4.2 detalha especificações técnicas do MCU utilizado; e a Seção 4.3 aprofunda com mais detalhes do processo de implementação, bem como as decisões de projeto tomadas ao longo do caminho. A Seção 4.4 discute alguns desafios inesperados enfrentados durante a implementação.

4.1 Projeto Proposto

Como declarado anteriormente, o objetivo deste trabalho é desenvolver uma ferramenta que, dado um modelo de rede neural pré-treinado, retorne um relatório com a latência de inferência medida no dispositivo (on-device). Ela foi pensada para ser útil tanto para integração com fluxos de trabalho de NAS quanto para uso na construção manual (hand-crafting) de arquiteturas, e essa flexibilidade guiou os princípios de projeto.

Para torná-la adequada ao uso em NAS, é importante tentar minimizar o tempo total de execução da ferramenta, para que todo o processo de NAS não se torne inviavelmente longo. Além disso, é imperativo que todo o processo seja completamente automático, já que qualquer intervenção humana necessária tornaria a ferramenta inutilizável para NAS. Para facilitar o uso na construção manual de modelos, os resultados são gerados em formato legível por humanos.

A Figura 4.1 mostra um fluxo de trabalho típico em que o B.A.L.A.S pode ser utilizado, onde a caixa pontilhada em vermelho delimita claramente o que a ferramenta de fato faz. Nesse arranjo, a ferramenta roda em um computador principal e no dispositivo embarcado alvo, que, neste caso, é a placa FRDM-MCXN947 (detalhes na Seção 4.2), conectada a ele via USB. Isso permite não apenas reprogramar o código embarcado, mas também comunicação entre o MCU e o computador principal. Para executar inferência na placa embarcada, escolheu-se o TensorFlow Lite for Microcontrollers, por ser o framework de TinyML mais usado e agnóstico a plataforma (HAN; SIEBERT, 2022), com suporte a diversos operadores. O B.A.L.A.S é executado invocando-se um script principal em Python, que integra uma mistura de aplicações recém-desenvolvidas e já existentes para realizar os passos necessários (detalhados na Seção 4.1.1). Escolheu-se Python pela facilidade de desenvolvimento e pela vasta disponibilidade de bibliotecas para utilidades de arquivos e comunicação, além de disponibilizar a biblioteca TensorFlow Lite para análise de modelos. Para flexibilidade, as entradas são passadas via argumentos de linha de comando, permitindo integração simples com pipelines automatizados e também uso manual. As opções propostas de linha de comando são:

• <model_path>: caminho para o arquivo .tflite ou .keras pré-treinado;

• <profiling_dataset>: caminho para o diretório contendo um arquivo para cada tensor de entrada de profiling em f32 (dados em formato de ponto flutuante de 32 bits);

• <results>: nome do arquivo CSV de saída com resultados.

Para reduzir o escopo do projeto, optou-se por suportar apenas modelos int8 pré-quantizados e totalmente quantizados. Isso se justifica pelo fato de a quantização int8 completa ser o tipo de quantização mais comum (como mencionado na Seção 2.3) e pelo fato de, em TinyML, a maioria dos modelos precisar ser quantizada para caber nos recursos limitados, o que se reflete no MLPerfTiny (BANBURY et al., 2021), cujas submissões da primeira rodada tinham, em sua maioria, modelos quantizados em int8.

4.1.1 Detalhes do Projeto

Para ajudar a compreender mais claramente a operação do B.A.L.A.S, o diagrama da Figura 4.2 mostra com mais detalhes os módulos internos e suas funcionalidades. Há três módulos de alto nível: Model Deployer, Model Profiler e o código desenvolvido para o Dispositivo Embarcado Alvo. Cada um é explicado abaixo.

O Model Deployer é o primeiro a executar. Ele é responsável por receber um modelo, realizar conversões adequadas, gerar, compilar e fazer deploy da aplicação que executará a inferência na placa alvo. Seus módulos são:

• Estimador de Tensor Arena: diferentes ferramentas e métodos foram analisados para encontrar um que, dado um modelo .tflite, determine quanta memória deve ser pré-alocada para a tensor arena no TensorFlow Lite for Microcontrollers. Ele deve evitar subestimar o valor, o que causa erro em tempo de execução, e superestimar o mínimo possível, para evitar usar mais RAM do que o necessário;

• Gerador de Código C++ Customizado: um script em Python que, usando o valor estimado do tamanho da tensor arena e a arquitetura do modelo analisada a partir do arquivo .tflite, edita um projeto C++ “template”, definindo constantes e gerando chamadas de função específicas por camada;

• Compilador e Deployer: um script em Python que invoca os métodos adequados de build e compilação para a placa utilizada.

Após o modelo ser implantado na placa, o Model Profiler é ativado. Ele segue uma metodologia similar ao MLPerfTiny (BANBURY et al., 2021), enviando, um a um, um lote de entradas de teste e, em seguida, lendo os resultados do tempo de inferência. Assim como no MLPerfTiny, apenas a inferência é temporizada, sem incluir quantização, desquantização e quaisquer passos de pré/pós-processamento. Por fim, o processo é concluído com a geração de relatórios de resultados.

A terceira parte do projeto é o projeto C++ “template” que implementa a inferência. Partes desse código são editadas pelo Model Deployer para torná-lo customizado ao modelo de entrada, e ele executa no MCU em paralelo ao Model Profiler, que roda no host. Sua funcionalidade é atuar como um servidor UART (Universal Asynchronous Receiver/Transmitter), aguardando entradas, executando inferência, medindo o tempo de inferência e retornando resultados. Esse projeto consiste das seguintes partes:

• Compilação do TensorFlow Lite for Microcontrollers: para acelerar o processo de compilação, a biblioteca TFLM é pré-compilada para o ARM Cortex-M33;

• Inicialização do modelo: código “boilerplate” padrão do TFLM para inicialização, deixando lacunas onde código específico do modelo deve ser injetado pelo Deployer;

• Leitura e escrita via UART: código usando o BSP (Board Support Package) específico do MCU implementa inicialização da UART, leitura e escrita;

• Temporizador: similar à UART, a funcionalidade de temporização é implementada usando drivers do fabricante;

• Inferência: uma vez feita a inicialização, executar inferência é tão simples quanto chamar uma função da biblioteca. A entrada precisa ser quantizada antes da inferência e desquantizada depois.

4.2 Placa de Desenvolvimento FRDM-MCXN947

A placa de desenvolvimento FRDM-MCXN947 é fabricada e vendida pela NXP Semiconductors. Ela contém um microcontrolador MCXN947, bem como outras facilidades de desenvolvimento, como conectores, conversores e pinos de fácil acesso. Mais importante, ela possui uma interface USB-C, que pode ser usada para alimentar a placa, gravar binários, depurar via SWD (Serial Wire Debug) e comunicar-se com um dos módulos UART (NXP, 2025).

O MCXN947 possui dois núcleos ARM Cortex-M33, ambos com frequência de clock de até 150 MHz. Um dos núcleos é o primário, com recursos adicionais como FPU (Floating Point Unit) e suporte a instruções vetoriais SIMD (Single Instruction Multiple Data) otimizadas. Ele tem dois bancos de flash de 1 MB cada, bem como 512 kB de SRAM (416 kB se códigos de correção de erro estiverem ativos). Também possui aceleradores específicos de hardware, como um acelerador DSP (Digital Signal Processing) e uma NPU (Neural Processing Unit). Entretanto, o escopo deste trabalho está limitado à CPU; assim, esses aceleradores não foram utilizados.

A Figura 4.3 ilustra os módulos de hardware presentes nesse microcontrolador.

Esta placa de desenvolvimento foi escolhida porque foi projetada com TinyML em mente, o que é evidenciado pela presença da NPU para acelerar IA na borda (edge AI). Além disso, trata-se de um MCU ARM Cortex-M, que, como mencionado anteriormente, é o mais difundido em pesquisas de TinyML.

4.3 Implementação

4.3.1 Pré-processamento de Dados para os Modelos

Para executar o Model Profiler, é necessário um pequeno conjunto de dados com N tensores de entrada em float32 para cada um dos modelos de benchmarking. Inspirado pela metodologia de (BANBURY et al., 2021), utilizou-se N = 10. Cada modelo usa um conjunto de dados diferente e espera um tipo diferente de pré-processamento aplicado à entrada, detalhado abaixo:

• Classificação de Imagens: este modelo utiliza o conjunto CIFAR-10, que foi baixado integralmente. As imagens do CIFAR-10 são representadas como valores uint8 32×32×3, enquanto os parâmetros de quantização do modelo mapeiam [0, 255] para a faixa int8 ([−128, 127]). Para manter uma interface unificada (considerando que todos os modelos recebem entradas float32 antes da quantização), os 10 tensores de profiling selecionados (um de cada classe) foram primeiro convertidos para float32 antes de serem salvos em arquivos binários;

• Visual Wake Words: este modelo utiliza uma versão reduzida e reamostrada (downsampled) do conjunto COCO chamada vw_coco2014_96, com cerca de 100 mil imagens 96×96×3 em formato JPEG, rotuladas como pessoa ou não-pessoa. O modelo espera tensores 96×96×3 de valores de pixel float32 normalizados no intervalo [0,0, 1,0]. Cinco imagens foram selecionadas aleatoriamente de cada rótulo; em seguida, foram decodificadas de JPEG para float32, normalizadas e salvas como arquivos binários;

• Keyword Spotting: este modelo utiliza o conjunto Speech Commands v2, composto por arquivos .wav de 1 segundo contendo palavras curtas faladas. Como entrada, o modelo espera um espectrograma MFCC 49×10 do áudio, portanto um script em Python foi escrito para converter as 10 entradas .wav selecionadas aleatoriamente para espectrogramas MFCC float32 a serem salvos como arquivos binários;

• Detecção de Anomalias: este modelo utiliza o conjunto ToyADMOS de sons de máquinas, mas apenas o subconjunto ToyCar. O conjunto completo possui dezenas de gigabytes, então somente o ToyCar foi baixado, contendo arquivos .wav de 10 segundos com sons do carro. Como entrada pré-quantização, o

modelo espera um espectrograma Mel 1×640 do som; assim, um script em Python foi escrito para converter 5 entradas normais selecionadas e 5 entradas anômalas selecionadas, com os espectrogramas float32 salvos como arquivos binários no conjunto de profiling.

Após esta etapa de pré-processamento, cada modelo ficou com seu diretório de profiling pronto para ser passado à ferramenta via argumento de linha de comando. Os tamanhos de entrada em float32 para cada modelo podem ser vistos na Tabela 4.1.

4.3.2 Projeto C++

Com o objetivo de receber entradas, executar inferência e medir seu tempo, um projeto em C++ deve ser configurado e implementado, representando o bloco “Target Embedded Device” na Figura 4.2. Este projeto foi inicialmente desenvolvido executando o Modelo de Classificação de Imagens e, depois, o código específico desse modelo foi substituído pelo Gerador de Código Customizado (Seção 4.3.5).

4.3.2.1 Compilação do TFLM

O primeiro passo foi compilar o TFLM em uma biblioteca .a especificamente para o Cortex-M33. Essa pré-compilação é crítica; caso contrário, a biblioteca teria de ser recompilada a cada execução, adicionando tempo a todo o processo automatizado de profiling. O TFLM oferece a opção de compilar usando instruções vetoriais otimizadas para chips ARM, o que acelera a inferência. Como essa opção seria usada em um cenário real onde latência deve ser minimizada, essa flag de compilação foi habilitada. A seguir estão os comandos usados para gerar a biblioteca compilada, bem como os headers do TFLM e de bibliotecas de terceiros:

```
make -f tensorflow/lite/micro/tools/make/Makefile \ TARGET=cortex_m_generic
TARGET_ARCH=cortex-m33 \ OPTIMIZED_KERNEL_DIR=cmsis_nn microlite
```

```
make -f tensorflow/lite/micro/tools/make/Makefile \ TARGET=cortex_m_generic
TARGET_ARCH=cortex-m33 \ OPTIMIZED_KERNEL_DIR=cmsis_nn list_library_headers \ >
tflm_headers.txt
```

```
make -f tensorflow/lite/micro/tools/make/Makefile \ TARGET=cortex_m_generic
TARGET_ARCH=cortex-m33 \ OPTIMIZED_KERNEL_DIR=cmsis_nn list_third_party_headers \ >
third_party_headers.txt
```

O passo final foi escrever um script em Python que recebe as listas de arquivos de header e os reorganiza em uma estrutura mais adequada para importação em um projeto C++. Tudo foi empacotado em um único diretório “tensorflow”, contendo a biblioteca .a compilada e todos os headers.

4.3.2.2 Configuração Básica do Projeto

Para desenvolver o código embarcado, utilizou-se o MCUXpresso IDE. Esta foi a escolha natural, pois oferece ferramentas para build, gravação (flash) e configuração da placa, bem como projetos de exemplo. Como ponto de partida, foi usado o exemplo “Hello World”. Uma configuração importante nesta etapa foi aumentar os tamanhos padrão de stack e heap descritos no script de linker para acomodar as grandes quantidades de dados usadas por modelos de ML.

4.3.2.3 Classe Genérica de Inferência de Modelo TFLM

A classe principal no projeto é MyModel, que, em seu construtor, é responsável por carregar os pesos do

modelo da memória flash e alocar todas as estruturas usadas pelo framework dentro da tensor arena pré-alocada (Figura 4.4). Ela também expõe um método para executar inferência, que quantiza o tensor de entrada, computa a inferência, desquantiza o resultado e o escreve na saída. Para facilitar o desenvolvimento, classes ModelInput e ModelOutput foram criadas como “wrappers” para arrays float32 alocados dinamicamente. Parâmetros que dependem do modelo incluem o tamanho da tensor arena, o número de operadores no resolver e a lista de operadores usados pelo modelo, como mencionado na Seção 2.6.

4.3.2.4 Configuração da UART

Em um microcontrolador, o estado padrão é que a maioria (se não todos) os periféricos esteja desabilitada e o desenvolvedor precise escrever código para habilitar e configurar cada periférico na inicialização. Isso é feito para economizar energia e usar apenas o que é necessário. Para economizar tempo de desenvolvimento, em vez de ler milhares de páginas de datasheet e configurar manualmente bits de registradores mapeados em memória, foram usadas as ferramentas de GUI do MCUXpresso IDE, que geram código de inicialização de periféricos conforme configurado.

Para a configuração da UART, como o objetivo é comunicar-se com a estação de trabalho via interface serial USB, era importante escolher o módulo UART e os pinos corretos, de modo que fossem aqueles conectados ao conector físico USB-C na placa. Seguindo o Manual de Referência, o periférico UART LPFLEXCOMM_4 foi usado e mapeado para os pinos P1_8 e P1_9. A Figura 4.5 mostra a tela de configuração com as demais configurações de UART utilizadas.

4.3.2.5 Configuração do Temporizador

Da mesma forma que a UART, um periférico de temporizador precisou ser configurado para medir o tempo de inferência. Uma diferença chave, porém, é que, para o temporizador, deve-se dar atenção especial ao roteamento do clock. Cada periférico pode ter sua fonte de clock selecionada, e o temporizador pode ser configurado para contar a cada T ciclos de clock; portanto, é crucial encontrar uma combinação adequada desses parâmetros.

Foi necessária uma granularidade de microssegundos para o timing de inferência; assim, usando a ferramenta de configuração de clock (Figura 4.6), um clock de 1 MHz foi roteado para o temporizador e configurado para contar a cada ciclo, efetivamente contando microssegundos.

4.3.3 Automação de Build e Deploy

Um dos objetivos desta ferramenta é ser totalmente automatizada; portanto, não se pode esperar que o usuário precise abrir a IDE e interagir com a GUI para compilar e gravar o projeto C++. A solução foi escrever dois scripts: um para compilar o projeto e outro para fazer deploy na placa.

Para compilação, seria possível escrever manualmente um Makefile usando a toolchain de cross-compilação arm-none-eabi. Entretanto, isso demandaria tempo considerável devido à complexidade do projeto e a todos os drivers adicionados via MCUXpresso IDE. A solução está na própria IDE: por ser baseada no Eclipse, ela oferece comandos de build “headless” via CLI, que permitem compilar um projeto do workspace com um único comando. O script de build foi então escrito com base no MCUXpresso IDE Command Line User Guide, tomando cuidado para garantir que um “make clean” não fosse executado a cada vez, economizando tempo ao evitar recompilar arquivos desnecessariamente.

O script de deploy também foi escrito com auxílio da IDE. O MCUXpresso oferece uma ferramenta

gráfica de flash que identifica placas NXP conectadas via USB e permite gravação fácil dos binários. Essa ferramenta também tem uma opção de exibir os comandos que executa; esses comandos foram então adaptados para um script, que é invocado logo após a compilação.

4.3.4 Estimativa de Tensor Arena

Como explicado na Seção 2.6.1, o TFLM exige um buffer estaticamente alocado e de tamanho fixo, onde aloca as estruturas usadas durante a inferência. Foram consideradas as seguintes opções para lidar com isso:

4.3.4.1 Tamanho fixo muito grande

Escolher um tamanho constante muito grande de tensor arena (por exemplo, 400 kB) para todos os modelos funcionaria para a maioria deles. O problema é que, se um modelo precisar de muito menos do que esse tamanho, há desperdício significativo de RAM. Por exemplo, um modelo que precise de apenas 200 kB de tensor arena, mas precise de 150 kB para seu tensor de entrada em float32, não caberia em 512 kB de RAM caso se usasse um tamanho fixo de 400 kB para todos os modelos; entretanto, ele poderia caber se houvesse uma estimativa melhor do tamanho da arena.

4.3.4.2 Tentativa e erro

Uma opção seria começar com um tamanho pequeno de tensor arena e tentar implantar o modelo: se não funcionar, tentar novamente com um tamanho maior e repetir até funcionar. O problema é que, embora encontre um tamanho adequado, cada tentativa envolve compilar e fazer deploy na placa, o que consumiria tempo demais para fazer benchmarking de um único modelo.

4.3.4.3 Ferramenta utilitária STM32

As ferramentas que acompanham o framework STM32Cube.AI também oferecem utilitários para auxiliar o uso do TFLM em placas STM32. Um desses utilitários, chamado “stm32tflm”, recebe como entrada um arquivo .tflite e escreve na saída padrão (STDOUT) quanto de flash o modelo ocupa e quanta RAM precisa ser alocada na tensor arena.

Para avaliar a acurácia das estimativas do stm32tflm, o seguinte procedimento foi executado para cada um dos quatro modelos de benchmark: primeiro, o método de tentativa e erro foi usado manualmente para encontrar o menor tamanho de tensor arena necessário; em seguida, a ferramenta STM32 foi usada, e a estimativa foi comparada ao menor tamanho encontrado empiricamente. Os resultados foram compilados na Tabela 4.2.

Observa-se que o stm32tflm produz boas estimativas do tamanho de tensor arena requerido, com três pequenas superestimativas e uma pequena subestimativa. A maior superestimativa entre os quatro modelos padrão é no modelo de Keyword Spotting, alocando 7,6% a mais do que o necessário. Para este trabalho, isso foi considerado aceitável; portanto, o stm32tflm foi o método escolhido para estimar a tensor arena. Contudo, nota-se que para um dos modelos houve subestimativa, o que faz o modelo não funcionar. A solução proposta para esse problema é apresentada na Seção 4.3.6.

4.3.5 Gerador de Código C++ Customizado

Como mencionado na Seção 2.6.1, embora o TFLM use um interpretador genérico, ainda é necessário algum código específico do modelo para que ele funcione. Este módulo é responsável por gerar e injetar esse código no projeto C++ em três etapas: gerar chamadas de função do resolver, gerar o header binário

do modelo e injetar o código no projeto.

4.3.5.1 Geração de chamadas do resolver

Para descobrir quais operadores (camadas) o modelo usa, o arquivo .tflite é analisado com a biblioteca Python do TFLite, a partir da qual os nomes das camadas são extraídos. Foi produzido um dicionário em Python que mapeia nomes de camadas do TFLite para seus respectivos nomes de funções no resolver. Esse dicionário foi gerado a partir da análise do header micro_mutable_ops_resolver.h do TFLM.

4.3.5.2 Geração do binário do modelo

Para inicialização do modelo na classe MyModel, é necessário incluir um header que contenha um array C com todos os dados binários do arquivo .tflite. Isso é gerado usando o utilitário de linha de comando xxd e editado para ter um nome de variável conhecido.

4.3.5.3 Injeção de código

Os arquivos que precisam de código customizado (model.cpp e model.h) possuem cópias que permanecem em um diretório de templates. Esses arquivos template têm tags “marcadoras” onde o tamanho da tensor arena, o número de operações no resolver e as chamadas de funções do resolver devem ser inseridos. Na etapa de injeção de código, os templates são primeiro copiados para o diretório do projeto C++ e, então, os valores gerados são inseridos via busca e substituição (find and replace). Essas cópias template são necessárias porque, caso contrário, após a primeira execução, as tags teriam sido sobrescritas.

4.3.6 Model Profiler

Após o modelo ser implantado na placa e ficar aguardando entradas de teste via UART, o Model Profiler é o módulo responsável por enviar essas entradas. Ele lê todos os arquivos binários float32 do diretório do conjunto de profiling informado, abre uma conexão serial com a interface USB do dispositivo, envia um tensor de entrada e, então, aguarda a resposta do MCU, que é o tempo de inferência em microssegundos.

Para lidar com o problema de subestimativas de tamanho da tensor arena, foi configurado um timeout de 10 segundos na leitura serial dos resultados de timing. Se ocorrer timeout, assume-se que a tensor arena era pequena demais, pois a aplicação C++ “trava” indefinidamente durante a construção da classe MyModel nesse caso. Então, o tamanho da tensor arena é incrementado em 5.000 bytes e o fluxo retorna à compilação, repetindo o processo com uma arena maior, como mostrado na Figura 4.7.

4.3.6.1 Relatório

A saída final da ferramenta é um relatório em CSV, aberto em modo append para permitir que resultados de múltiplos modelos sejam escritos no mesmo arquivo. Cada seção relevante da execução do processo completo também é temporizada e adicionada ao relatório, com o objetivo de produzir dados que ajudem a analisar quanto tempo o processo de profiling levou. Os campos adicionados ao relatório são:

• model: nome do arquivo do modelo .tflite usado;

• arena_size: tamanho final da tensor arena;

• macs: número de MACs do modelo;

• avg_inference_time_us: tempo médio das inferências em microssegundos;

• std_inference_us: desvio padrão das inferências;

• arena_time_ms: tempo do stm32tflm em milissegundos;

• mac_time_ms: tempo do cálculo de MACs em milissegundos;

• cpp_codegen_time_ms: tempo do gerador de código C++ em milissegundos;

• compile_time_ms: tempo de compilação do projeto C++ em milissegundos;

• deploy_time_ms: tempo de deploy na placa em milissegundos;

• profiling_time_ms: tempo do profiling em milissegundos;

• attempts: número de tentativas necessárias para obter o tamanho correto de tensor arena.

4.4 Desafios de Implementação

Esta seção busca documentar alguns desafios inesperados durante a implementação. Seu principal objetivo é registrar essas dificuldades e suas soluções, para que outros desenvolvedores de TinyML que desejem fazer trabalho similar não precisem passar pelas mesmas etapas.

4.4.1 Flag de compilação do TFLM

Inicialmente, após a configuração do projeto com o TFLM compilado e seus headers organizados, o programa travava durante a inicialização do modelo, mesmo com modelos de teste muito simples. Após algumas semanas tentando resolver esse problema bloqueante e analisando o código-fonte do TFLM, identificou-se que a ausência da flag de build “TF_LITE_STATIC_MEMORY” era a causa do problema. Ao adicioná-la nas configurações do projeto na IDE, o problema foi resolvido.

4.4.2 Pinos de UART

Determinar exatamente quais pinos configurar para que leituras e escritas UART passassem pelo conector físico USB-C foi outro desafio. Lendo o Manual de Referência, entendeu-se inicialmente que os pinos P1_20 e P1_21 deveriam ser usados, mas isso não gerava saída na interface USB. Somente após uma análise mais aprofundada do manual é que se localizou a informação de uso dos pinos P1_8 e P1_9, que então foi testada com sucesso.

4.4.3 Ferramenta de Configuração de Clock do MCUXpresso

Outro desafio foi aprender a usar o MCUXpresso IDE, que é bastante completo em recursos, mas possui curva de aprendizado para algumas configurações. A ferramenta de configuração de clock foi particularmente pouco intuitiva. Nela, é possível selecionar quais fontes de clock, com quais multiplicadores, são roteadas para os periféricos. Sem essa configuração, nem o temporizador nem a UART funcionavam.

Entretanto, observou-se que o sinal de clock do FLEXCOMM4 (para UART) não estava habilitado, ainda que o multiplexador do sinal selecionasse uma fonte de clock adequada. Clicar com o botão direito no sinal de clock não apresentava opções, e em tutoriais online isso parecia funcionar diretamente. Após dias enfrentando esse problema, encontrou-se uma solução pouco intuitiva: o botão para habilitar o sinal de clock só aparecia ao clicar com o botão direito na mensagem de erro associada ao sinal.